Système d'exploitation
CM8 : numéro de port et *sockets* UDP

Pierre Rousselin
rousselin@univ-paris13.fr

Université Sorbonne Paris Nord
L2 informatique et double-licence

mars 2026

Rappels sur les couches

Couche transport

Couche application (brève introduction)

Un exemple minimal de protocole applicatif : *INCP*

UDP

Les *sockets* de Berkeley

Création, configuration et utilisation de `sockets`

Exemple de client et serveur INCP au-dessus d'UDP

Rappels sur les couches

Couche transport

Couche application (brève introduction)

Un exemple minimal de protocole applicatif : *INCP*

UDP

Les *sockets* de Berkeley

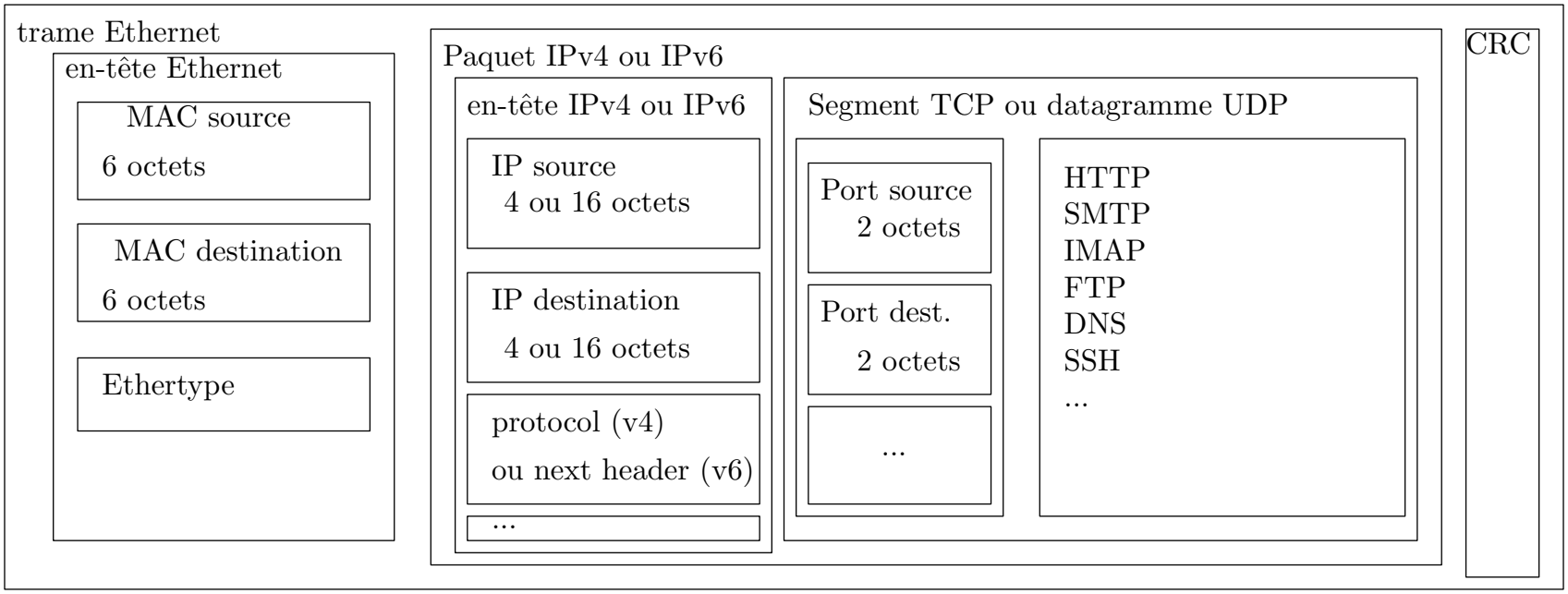
Création, configuration et utilisation de `sockets`

Exemple de client et serveur INCP au-dessus d'UDP

Couches basses

- La couche physique fait transiter des bits dans un milieu, d'une interface réseau source à une interface réseau destination.
- La couche liaison fait transiter des trames entres nœuds voisins sur le même support (réseau local).
- La couche réseau achemine des paquets entre machines...
- ... sans garantie que les paquets arrivent dans l'ordre où ils ont été émis ...
- ... ni même qu'ils arriveront vraiment (des paquets se perdent).

Encapsulations



Segment TCP ou datagramme UDP dans un paquet IP (v4 ou v6) dans une trame Ethernet.

Rappels sur les couches

Couche transport

Couche application (brève introduction)

Un exemple minimal de protocole applicatif : *INCP*

UDP

Les *sockets* de Berkeley

Création, configuration et utilisation de `sockets`

Exemple de client et serveur INCP au-dessus d'UDP

Couche transport : rôle

- La couche transport permet de faire communiquer des processus distants.
- S'il le faut (et il le faut souvent), elle est donc responsable de :
  - remettre dans l'ordre les paquets ;
  - détecter des erreurs dans les paquets ;
  - renvoyer les paquets perdus.
- Mise en œuvre par le système d'exploitation.

Numéro de port

- Autre problème : comment savoir à quel processus est destiné un paquet ?
- Le processus utilise un numéro de port : un entier non signé sur 16 bits (de 0 à 65535). Dans les en-têtes UDP et TCP, ce numéro de port est gros-boutiste.

Protocoles de la couche transport : UDP

Il y a 2 protocoles archi-dominants dans la couche transport.
L'un des deux est UDP (*User Datagram Prococol*) :

- sans connexion ;
- échange de petits paquets avec un début et une fin appelés *datagrammes* dont la taille est connue,
- avec *un début et une fin* (préserve les bords),
- sans garantie sur l'ordre de réception des paquets, ni même sur leur réception tout court ;
- surcouche minimale au-dessus de IP ;
- petite somme de contrôle pour vérifier que le paquet n'a pas été corrompu pendant le transport.

Protocoles de la couche transport : TCP

Il y a 2 protocoles archi-dominants dans la couche transport.
L'autre est TCP (*Transmission Control Protocol*) :

- établissement d'une connexion entre deux processus,
- transmission d'un flux de données,
- sans préservation des bords (les tailles des différents segments du flux sont imprévisibles) ;
- fiable, les paquets sont remis dans l'ordre et ceux qui ne sont pas reçus sont redemandés ;
- protocole très compliqué, lourd, c'est la bête de somme d'Internet

Communication entre deux processus

Pour que deux processus (distants) puissent se parler :

- Il faut qu'ils utilisent le même protocole transport.
- Il faut que le système d'exploitation de chaque hôte implémente ce protocole transport (en pratique c'est le cas).
- Il faut que le processus source du message connaisse le port utilisé par le processus destinaire.
- Si ce processus destinataire désire répondre au premier, il doit aussi connaître le port [1] de la machine qui lui a envoyé le message : heureusement, cela fait partie de l'en-tête du message.

1. et l'adresse IP de la machine mais ça c'est dans la couche réseau

Exemple (1)

- Un processus A sur un hôte H_A utilise le protocole UDP sur le port $P_A = 20000$.
- Il souhaite envoyer le datagramme de 5 octets `"Lol!\n"` à un processus B sur un hôte H_B.

Exemple (1)

- Un processus A sur un hôte H_A utilise le protocole UDP sur le port $P_A = 20000$.
- Il souhaite envoyer le datagramme de 5 octets `"Lol!\n"` à un processus B sur un hôte H_B.
- Le processus B doit utiliser aussi le protocole UDP, avec un certain numéro de port, disons $P_B = 30000$.

Exemple (1)

- Un processus A sur un hôte H_A utilise le protocole UDP sur le port $P_A = 20000$.
- Il souhaite envoyer le datagramme de 5 octets `"Lol!\n"` à un processus B sur un hôte H_B.
- Le processus B doit utiliser aussi le protocole UDP, avec un certain numéro de port, disons $P_B = 30000$.
- Le processus A doit connaître, d'une façon ou d'une autre ce numéro de port ! Et bien sûr, l'adresse IP I_B de l'hôte H_B. Supposons que c'est le cas.
- Le processus A demande au système d'exploitation de H_A d'envoyer `"Lol!\n"` en UDP à l'adresse I_B sur le port P_B.

Exemple (2)

- La couche réseau (IP) se charge ensuite d'acheminer le paquet de H_A à H_B. On suppose que le paquet arrive bien.
- Le système d'exploitation de H_B traite le paquet reçu et voit que c'est de l'UDP sur port 30000. Il transmet donc le datagramme au processus B... qui fera ce qu'il doit faire (protocole applicatif) avec.
- Si B souhaite répondre `"Rofl!\n"` à A il peut le faire car il connaît l'adresse I_A de H_A (fait partie de l'en-tête IP) et aussi le port P_A de A (fait partie de l'en-tête UDP).

Port éphémère

- L'exemple précédent montre une asymétrie entre A et B.
- A doit connaître à l'avance l'adresse I_B et le port P_B...
- ... alors que B reçoît ces informations dans le datagramme envoyé par A.
- Le processus B doit donc avoir un port fixé et connu...
- ... tandis que celui de A peut être choisi par le système d'exploitation de P_A : on parle de *port éphémère*.

Exemple sous Linux avec `socat` (1)

Hôte H_B souhaitant recevoir des datagrammes sur son réseau.

- Couche liaison et interfaces réseau (rappel) :

```
rousselin@v201-1:~$ ip link
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP>
link/ether 56:45:52:43:41:10 brd ff:ff:ff:ff:ff:ff
```

- Couche réseau (IP) :

```
rousselin@v201-1:~$ ip address show dev eth0
2: eth0@if4: <BROADCAST,MULTICAST,UP,LOWER_UP>
    inet 10.11.201.1/16 brd 10.11.255.255 scope global eth0
```

- Création d'un processus B qui attend des datagrammes sur le port 30000 puis les affiche sur le terminal :

```
rousselin@v201-1:~$ socat -dd udp-recv:30000 -
listening on UDP AF=2 0.0.0.0:30000
```

- Adresse `0.0.0.0` pour dire que l'on accepte les paquets quelle que soit l'adresse de destination (pourvu, bien sûr, qu'elle soit valide pour nous), par exemple `10.11.201.1` mais aussi `127.0.0.1` (*loopback*).

Exemple sous Linux avec `socat` (2)

Hôte H_A souhaitant envoyer un datagramme sur son réseau.

- Couches liaison et réseau :

```
rousselin@v201-6:~$ ip link
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP>
    link/ether 56:45:52:43:41:15 brd ff:ff:ff:ff:ff:ff
rousselin@v201-6:~$ ip address show dev eth0
2: eth0@if4: <BROADCAST,MULTICAST,UP,LOWER_UP>
    inet 10.11.201.6/16 brd 10.11.255.255 scope global eth0
```

- Création d'un processus A souhaitant communiquer avec B

```
rousselin@v201-1:~$ socat -dd udp-sendto:10.11.201.1:30000 -
```

Exemple sous Linux avec `socat` (3)

- Processus A :

```
rousselin@v201-6:~$ socat -dd udp-sendto:10.11.201.1:30000 -
Lol!
local address: AF=2 0.0.0.0:39169
```

- Processus B :

```
rousselin@v201-1:~$ socat -dd udp-recv:30000 -
received packet with 5 bytes from AF=2 10.11.201.6:39169
Lol!
```

- Port éphémère utilisé par A : 39169.

Rappels sur les couches

Couche transport

Couche application (brève introduction)

Un exemple minimal de protocole applicatif : *INCP*

UDP

Les *sockets* de Berkeley

Création, configuration et utilisation de `sockets`

Exemple de client et serveur INCP au-dessus d'UDP

Questions en suspens

- Comment connaître le port associé à un certain service ?
- Comment connaître l'adresse IP d'une machine offrant ce service ? (On verra plus tard avec le DNS).
- Comment se faire comprendre du processus offrant ce service ? Comment comprendre sa réponse ?

Exemple HTTP (1)

Un utilisateur utilisant un processus `firefox` souhaite télécharger la page web `www.perdu.com/index.html`. On suppose que `firefox` sait que la machine hébergeant ce service a pour adresse IP `172.67.133.176`.

- Le protocole applicatif utilisé est HTTP (*Hypertext Transfer Protocol*).
- Ce protocole applicatif est, par convention, associé au port 80 au-dessus du protocole transport TCP.
- Le processus `firefox` sait convertir la demande de l'utilisateur en une *requête HTTP*.
- Le système d'exploitation fournit à `firefox` un port éphémère pour cet échange, ainsi que les moyens (couche transport) de communiquer avec le processus (serveur HTTP) qui utilise le port 80 avec TCP sur la machine `172.67.133.176`.

Exemple HTTP (2)

- Le processus `firefox` est capable de comprendre la réponse qui lui est donnée car elle suit le protocole bien défini HTTP.
- Enfin, `firefox` sait afficher la page car c'est du HTML (*Hypertext Markup Language*) et l'utilisateur sourit car il est heureux.

Le protocole HTTP est l'un des milliers de *protocoles applicatifs* d'Internet.

Association port ↔ protocole applicatif : les *well-known ports*

Les ports de 0 à 1023 sont ceux utilisés pour les services les plus connus. Par exemple

- 22 : ssh (le plus souvent au-dessus de TCP)
- 80 : http (le plus souvent au-dessus de TCP)
- 53 : dns (le plus souvent au-dessus d'UDP)
- 143 : imap (le plus souvent au-dessus de TCP)
- 67 : dhcp (le plus souvent au-dessus d'UDP)
- 443 : https (le plus souvent au-dessus de TCP)

Le système d'exploitation ne permet qu'à `root` de lancer des processus qui écoutent (lisent) sur ces ports.

Informations sur les ports

- La command `ss` (pour *socket statistics*) sur Linux est le standard de fait pour connaître la liste des ports utilisés par les processus.
- Le fichier `/etc/services` contient la liste des associations port $\leftrightarrow$ service.
- La commande `nmap` permet de voir les ports en écoute sur un serveur.

Rappels sur les couches

Couche transport

Couche application (brève introduction)

Un exemple minimal de protocole applicatif : *INCP*

UDP

Les *sockets* de Berkeley

Création, configuration et utilisation de `sockets`

Exemple de client et serveur INCP au-dessus d'UDP

Incrementation Protocol

INCP pour *Incrementation Protocol* est le protocole applicatif le plus bête du monde (©Pierre Rousselin).

- Un client envoie un entier à un serveur ;
- Le serveur reçoit cet entier, l'incrémente et renvoie l'entier incrémenté au client.

Problèmes

- Quel port ? Quel protocole transport sous-jacent ?
- Entier ? Signé ? Quelle taille ?
- Entier ? Quel boutisme ?

Problèmes

- Quel port ? Quel protocole transport sous-jacent ?
- Entier ? Signé ? Quelle taille ?
- Entier ? Quel boutisme ?

Même une chose aussi simple a besoin de spécification.

INCP

- au dessus de TCP ou d'UDP
- sur le port 1234
- Entier non signé sur 4 octets
- En *network byte order* (gros-boutiste).

INCP

- au dessus de TCP ou d'UDP
- sur le port 1234
- Entier non signé sur 4 octets
- En *network byte order* (gros-boutiste).

On voit qu'au minimum, un protocole applicatif doit fixer :

- le port utilisé par le serveur et le ou les protocoles transports utilisés
- le format des données
- et plus généralement la façon dont se déroulent les échanges (ici un client envoie un entier, le serveur lui répond et l'échange est terminé).

INCP

- au dessus de TCP ou d'UDP
- sur le port 1234
- Entier non signé sur 4 octets
- En *network byte order* (gros-boutiste).

On voit qu'au minimum, un protocole applicatif doit fixer :

- le port utilisé par le serveur et le ou les protocoles transports utilisés
- le format des données
- et plus généralement la façon dont se déroulent les échanges (ici un client envoie un entier, le serveur lui répond et l'échange est terminé).

Les protocoles applicatifs sont standardisés (avec le reste des protocoles TCP/IP) dans des documents publics appelés RFC (*request for comments*).

Rappels sur les couches

Couche transport

Couche application (brève introduction)

Un exemple minimal de protocole applicatif : *INCP*

UDP

Les *sockets* de Berkeley

Création, configuration et utilisation de `sockets`

Exemple de client et serveur INCP au-dessus d'UDP

Introduction à UDP

- Signifie *User Datagram Protocol*.
- Les deux mots sont importants :
- *User :* faites ce que vous voulez avec !
- *Datagram :* On envoie ou reçoit des *datagrammes*, c'est-à-dire des petits messages avec un début et une fin, pas un flux d'octets.
- Couche minimale au-dessus d'IP : numéro de port source, numéro de port destination, somme de contrôle et c'est tout !

En-tête UDP

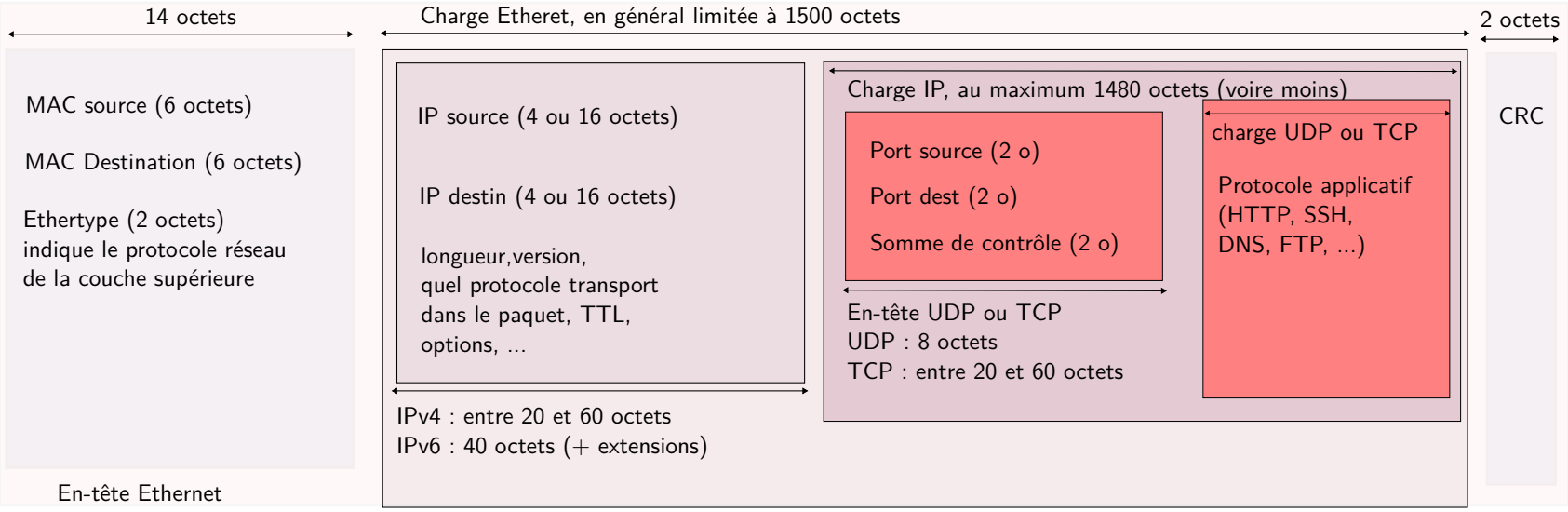
UDP Datagram Header Format

Bit #	0 – 7	8 – 15	16 – 23	24 – 31
0	Source Port		Destination Port	
32	Length		Header and Data Checksum	

- Ports car c'est un protocole de la couche transport (communication entre deux processus).
- Longueur sur deux octets donc limite dure à 65535 moins la taille de l'en-tête (en pratique, beaucoup moins la plupart du temps)
- Somme de contrôle pour détecter certaines erreurs (utilisée ou non).

UDP sur IPv4 sur Ethernet

En pratique, le protocole de la couche liaison est sans doute ethernet et la taille des trames est limitée la plupart du temps à 1500 octets.



Pour UDP, taille du datagramme limitée à 1472 octets sur la plupart des réseaux (en IPv4). Limite dure : le champ *length* de IP est déjà sur 2 octets donc $65535 - 20 - 8 = 65507$.

Rappels sur les couches

Couche transport

Couche application (brève introduction)

Un exemple minimal de protocole applicatif : *INCP*

UDP

Les *sockets* de Berkeley

Création, configuration et utilisation de `sockets`

Exemple de client et serveur INCP au-dessus d'UDP

Couche transport et Unix

Pour un processus Unix

- recevoir des données de l'extérieur correspond toujours à lire dans un fichier ;
- envoyer des données vers l'extérieur correspond toujours à écrire dans un fichier.

Couche transport et Unix

Pour un processus Unix

- recevoir des données de l'extérieur correspond toujours à lire dans un fichier ;
- envoyer des données vers l'extérieur correspond toujours à écrire dans un fichier.

Il est donc évident qu'un système Unix va offrir les services de la couche transport par le biais de fichiers.

Couche transport et Unix

Pour un processus Unix

- recevoir des données de l'extérieur correspond toujours à lire dans un fichier ;
- envoyer des données vers l'extérieur correspond toujours à écrire dans un fichier.

Il est donc évident qu'un système Unix va offrir les services de la couche transport par le biais de fichiers.
Ces fichiers sont les *sockets* de Berkeley (l'université californienne où ils ont été inventés et implémentés dans un Unix).

signification de *socket* en anglais

- *eye socket* : cavité occulaire
- *tooth socket* : cavité de la mâchoire dans laquelle la dent est ancrée
- *electrical socket* : prise electrique

La métaphore signifie donc que le processus va « se brancher à une prise » (un point terminal du réseau) à travers laquelle il pourra communiquer avec d'autres processus.

Rappels sur les couches

Couche transport

Couche application (brève introduction)

Un exemple minimal de protocole applicatif : *INCP*

UDP

Les *sockets* de Berkeley

Création, configuration et utilisation de `sockets`

Exemple de client et serveur INCP au-dessus d'UDP

Appel système `socket`

- Citation du `man socket`
  socket - create an endpoint for communication

  Extrémité de communication pour un *processus*.
- Canal potentiellement bidirectionnel (un processus peut y lire et y écrire) pour la communication entre processus :
  - sur la même machine (sockets Unix, `AF_UNIX`) ou
  - sur des machines potentiellement différentes connectées sur un même réseau, avec protocole réseau IPv4 (sockets ipv4, `AF_INET`) ou
  - sur des machines potentiellement différentes connectées sur un même réseau, avec protocole réseau IPv6 (sockets IPv6, `AF_INET6`).
  - `AF` veut dire *address family*.

Types de sockets

Un socket peut être de type :

- `SOCK_RAW` pour un accès « brut » au réseau (paquets IP) [2]
- `SOCK_STREAM` pour un *flux* de données bidirectionnel et fiable, bref... TCP
- `SOCK_DGRAM` pour des *datagrammes*, bref... UDP

2. il faut être `root` pour pouvoir créer ces sockets

Créer un socket

```
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
```

- Premier argument : la famille d'adresse. Pour nous, seulement `AF_INET` ou `AF_INET6`
- Deuxième argument : le type du socket. Pour nous, seulement `SOCK_STREAM` pour TCP et `SOCK_DGRAM` pour UDP.
- Troisième argument : toujours 0 car on se restreint à TCP et UDP.
- Valeur de retour : le descripteur de fichier, car un socket est un fichier !

Exemples

- Socket TCP/IPv4

```
int sock = socket(AF_INET, SOCK_STREAM, 0);
if (sock < 0) {
    perror("socket");
    exit(1);
}
```

- Socket UDP/IPv6

```
int sock = socket(AF_INET6, SOCK_DGRAM, 0);
if (sock < 0) {
    perror("socket");
    exit(1);
}
```

Appel système `bind`

Une fois le *socket* créé, il faut, au moins côté serveur :

- préciser le port utilisé par le serveur pour recevoir les messages ;
- **si besoin,** restreindre la réception des messages à ceux destinés à une certaine adresse IP.
- Par exemple, un serveur pourrait ne vouloir recevoir que les messages venant de l'interface *loopback* et se restreindre aux adresses `127.0.0.1/8`.
- Mais le plus souvent, le *socket* d'un serveur sera attaché à **toutes** les interfaces réseaux de la machine.
- Enfin, en général, il n'est pas nécessaire de `bind` explicitement un *socket* côté client.

Appel système `bind`, en pratique

```
#include <sys/socket.h>
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- Premier argument : le socket en question ;
- Deuxième argument : l'adresse en mémoire d'une structure contenant les informations (port, adresses et peut-être d'autres) nécessaires
- Troisième argument : la taille de cette structure (les adresses IPv4 et IPv6 n'ont pas la même taille donc il faut préciser...) ;
- Valeur de retour : 0 si ok, -1 sinon (avec `errno` mis à jour).

Les sockaddr c'est l'horreur (IPv4)

- Le second paramètre de `bind` est de type `struct sockaddr`.
- Il est destiné essentiellement à contenir une adresse IP (v4 ou v6) et un numéro de port.
- Le problème est que ces adresses sont très différentes... on a donc une structure « abstraite » (et assez inutile) :

```
struct sockaddr {
    sa_family_t sa_family;
    char        sa_data[14];
};
```

- Pour IPv4, la structure concrète (qui sera ensuite castée) est

```
struct sockaddr_in {
    sa_family_t    sin_family; /* address family: AF_INET */
    in_port_t      sin_port;   /* port in network byte order */
    struct in_addr sin_addr;   /* internet address */
};
/* Internet address. */
struct in_addr {
    uint32_t       s_addr;     /* address in network byte order */
};
```

- Voir `man 7 ip`.

Les sockaddr c'est l'horreur (IPv6)

```
struct sockaddr_in6 {
    sa_family_t     sin6_family;   /* AF_INET6 */
    in_port_t       sin6_port;     /* numéro de port */
    uint32_t        sin6_flowinfo; /* information de flux IPv6 */
    struct in6_addr sin6_addr;     /* adresse IPv6 */
    uint32_t        sin6_scope_id; /* Scope ID (nouveauté 2.4) */
};

struct in6_addr {
    unsigned char   s6_addr[16];   /* adresse IPv6 */
};
```

Voir `man ipv6`.

Les sockaddr en pratique

Quelques pratiques courantes :

- Utiliser la fonction

  ```
  #include <arpa/inet.h>
  int inet_pton(int af, const char *src, void *dst);
  ```

  pour passer d'une représentation sous forme de chaîne de caractère à une `struct in_addr` (pour IPv4) ou une `struct in6_addr` (pour IPv6) directement dans le bon boutisme.
- Utiliser `htons` pour convertir le numéro de port, si besoin, en gros-boutiste.
- Utiliser les constantes `INADDR_ANY` (`0.0.0.0`, n'importe quelle interface IPv4 du serveur), `INADDR_LOOPBACK` (`127.0.0.1`) avec `htonl`.
- Pour les `sockaddr` distantes, utiliser `getaddrinfo`.

Lire dans un socket UDP

Appel système bloquant par défaut :

```
ssize_t recvfrom(int sockfd, void *buf, size_t len,
  int flags, struct sockaddr *src_addr, socklen_t *addrlen);
```

- **sockfd** descripteur de fichier du *socket*
- **buf** à quelle adresse mémoire du processus écrire le datagramme reçu
- **len** combien d'octets au maximum on peut recevoir
- **flags** 0 pour l'instant
- **src_addr** pointeur vers une `sockaddr` que le système remplit avec l'adresse et le port de l'expéditeur, pour pouvoir éventuellement lui répondre
- **addr_len** pointeur vers un entier (attention, paramètre lecture/écriture), en lecture, la taille de ce qui est pointé par `src_addr`, en écriture la taille de la `sockaddr` reçue dans le message.

Valeur de retour Nombre d'octets effectivement reçus, ou -1 en cas d'erreur.

Écrire dans un `socket` UDP

Appel système bloquant par défaut :

```
ssize_t sendto(int sockfd, const void *buf, size_t len,
  int flags, const struct sockaddr *dest_addr, socklen_t addrlen);
```

- `sockfd` descripteur de fichier du *socket*
- `buf` adresse du premier octet (dans la mémoire du processus) du message à envoyer
- `len` longueur du message, en octets
- `flags` 0 pour l'instant
- `src_addr` pointeur vers une `sockaddr` qui contient l'adresse et le port du destinataire.
- `addr_len` taille de la `sockaddr` précédente

Valeur de retour Nombre d'octets effectivement écrits (en général, pour UDP, autant que `len`), ou -1 en cas d'erreur.

Rappels sur les couches

Couche transport

Couche application (brève introduction)

Un exemple minimal de protocole applicatif : *INCP*

UDP

Les *sockets* de Berkeley

Création, configuration et utilisation de `sockets`

Exemple de client et serveur INCP au-dessus d'UDP

Le protocole applicatif INCP

INCP pour *Incrementation Protocol* est le protocole applicatif le plus bête du monde (©Pierre Rousselin).

- Un client envoie un entier non signé de 4 octets en *network byte order* à un serveur ;
- Le serveur reçoit cet entier, l'incrémente et renvoie l'entier incrémenté au client.
- Le protocole INCP utilise le port 1234 au-dessus de TCP ou d'UDP.

Création et configuration du `socket` (client)

```
/* 1. Création d'une socket ipv4 udp */
int sock = socket(AF_INET, SOCK_DGRAM, 0);
if (sock < 0) {
    perror("socket");
    exit(2);
}
```

C'est tout (pas besoin de `bind`).

Création et configuration du socket (serveur)

```
#define PORT_INCP 1234
int sock = socket(AF_INET, SOCK_DGRAM, 0);
if (sock < 0) {
    perror("socket");
    exit(2);
}
/* 2. Création de la sockaddr locale */
struct sockaddr_in sa = { .sin_family = AF_INET, /* IPv4 */
              /* Port : 1234 converti si besoin en
               * gros-boutiste */
              .sin_port = htons(PORT_INCP),
              /* Adresse : n'importe laquelle sur laquelle
               * on reçoit de l'UDP sur port 1234
               * (en fait c'est 0.0.0.0), convertie si
               * besoin en gros-boutiste (même si ici ça
               * ne change rien). */
              .sin_addr.s_addr = htonl(INADDR_ANY) };
/* Optionnel : faire en sorte de pouvoir réutiliser l'adresse
 * et le port sans attendre après la fin du serveur. */
int opt = 1;
setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(int));
```

Création et configuration du `socket` (serveur, suite)

Maintenant, on peut attacher la `sockaddr` locale au *socket* pour que le serveur reçoive ce qui arrive en UDP, port 1234.

```
/* 3. Attacher le socket à la sockaddr locale */
if (bind(sock, (struct sockaddr *) &sa, sizeof(sa)) < 0) {
    perror("bind");
    exit(3);
}
```

C'est important de tester la valeur de retour de `bind` car ça peut mal se passer (par exemple : *Port already in use*).

Le serveur attend une requête

```
/* 4. Réception d'une requête INCP */
/* Il faut absolument une sockaddr pour enregistrer son adresse IP
 * son port */
struct sockaddr_in addr_clt;
socklen_t sl = sizeof(struct sockaddr_in);

uint32_t n;
/* Attendre la réception d'un message d'au plus 4 octets dans sock
 * mettre ces octets dans n
 * sans flag (0)
 * mettre l'adresse et le port de l'expéditeur dans addr_clt
 * et la taille de cette sockaddr dans sl */
ssize_t s = recvfrom(sock, &n, 4, 0,
    (struct sockaddr *) &addr_clt, &sl);
```

- point technique embêtant : le *cast* en `(struct sockaddr *)`
- Le point crucial est que le serveur sait que le message fait 4 octets car c'est **spécifié par le protocole applicatif.**

Le client remplit une sockaddr distante pour envoyer un message au serveur

On suppose que argv[1] est l'adresse IPv4 d'un serveur INCP.

```
/* 2. Création de la sockaddr distante */
struct sockaddr_in sa = { .sin_family = AF_INET, /* IPv4 */
                /* hotns : Host To Network Short pour
                 * convertir si besoin un entier sur 16 bits
                 * en gros-boutiste */
                .sin_port = htons(PORT_INCP)
            };

/* "Internet Presentation to Numeric" convertit une représentation
 * d'adresse IP (v4 ou v6) en numérique (gros-boutiste) */
if (inet_pton(AF_INET, argv[1], &sa.sin_addr) != 1) {
    fprintf(stderr, "adresse ipv4 non valable\n");
    exit(1);
}
/* À cause d'IPv4 versus IPv6 il faut souvent envoyer ou recevoir
 * la taille de la sockaddr */
socklen_t sl = sizeof(struct sockaddr_in);
```

Le client envoie sa requête au serveur

```
/* 4. Échange avec le serveur */
/* 4.1 Construction de la requête INCP (couche application) */
uint32_t n;
/* L'utilisateur entre son entier dans le terminal */
if (scanf("%" SCNu32, &n) < 1) {
    fprintf(stderr,
        "Erreur fatale : un entier positif était attendu.");
    return 1;
}
n = htonl(n); /* conversion si besoin en gros-boutiste */

/* 4.2 Envoi de la requête dans le socket */
/* Remarque : c'est la première fois que le socket est utilisé,
 * c'est à ce moment que le noyau choisit un port éphémère
 * et le lie à ce socket.
 * Envoyer, à travers sock, les 4 octets commençant à l'adresse &n,
 * sans flag (0), au couple (adresse IP, port distant) renseigné
 * dans sa, de taille sl octets */
int status = sendto(sock, &n, 4, 0, (struct sockaddr *) &sa, sl);
if (status == -1) {
    perror("sendto");
    return 2;
}
```

Le serveur reçoit la réponse du client

Le système d'exploitation du serveur le réveille lorsqu'un datagramme est reçu pour UDP, port 1234. Puis le serveur traite la requête du client.

```
/* 5. Traitement de la requête (protocole applicatif) */
n = ntohl(n); /* si besoin convertir en boutisme machine */
n = n + 1;    /* INCRÉMENTATION ! */
n = htonl(n); /* si besoin, remettre en gros-boutisme */
/* 6. Envoi de la réponse au client */
s = sendto(sock, &n, 4, 0, (struct sockaddr *) &addr_clt, sl);
/* 7. Ménage */
close(sock);
```

Réception du datagramme par le client

```
/* 4.3 Réception de la réponse du serveur */
ssize_t nb_bytes = recvfrom(sock, &n, 4, 0, (struct sockaddr *) &sa, &s
if (nb_bytes < 4) {
    /* Le protocole INCP interdit que le serveur réponde par autre
     * chose qu'un message de 4 octets, il y a une erreur */
    perror("recvfrom");
    return 3;
}
/* 4.4 Exploitation du message (couche application) */
n = ntohl(n); /* conversion si besoin dans le boutisme de la machine */
printf("%" PRIu32 "\n", n);
/* 5. Ménage */
close(sock);
```

Compilation et exécution

- Il y a **deux** programmes.
- Le client sur un hôte :

```
$ gcc -Wall client_udp_incp.c -o clt_incp
```

- Le serveur, éventuellement sur un autre hôte :

```
$ gcc -Wall serveur_udp_incp.c -o srv_incp
```

- Le serveur est lancé (pas besoin de `sudo` car le port est ≥ 1024) :

```
$ ./srv_incp
```

- Le client est lancé avec comme argument l'adresse IP d'un serveur en notation habituelle, par exemple ici en boucle locale :

```
$ ./clt_incp 127.0.0.1
42
43
```

- Et voilà !

Critiques

- D'abord le serveur devrait être dans une boucle infinie pour servir des clients pour l'éternité.
- Il est fréquent (mais pas ici...) que le traitement de la requête prenne du temps. Le serveur doit alors utiliser un *thread* par client à servir.
- C'est le protocole applicatif le plus basique possible, on va en voir de plus compliqués, mais l'essentiel des problèmes techniques est vu (au moins pour UDP).